



UITs
UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

PROJECT REPORT

ON

CAR RACING — OBSTACLE DODGE DRIVE

(Implemented Using OpenGL, C++)

Department : CSE

Course Code : CSE 358

Course Title : Computer Graphics and Multimedia Lab

Program : BSc in Computer Science & Engineering

Prepared by:

1. Name: Md. Tanvir Rahman

ID: 0432320005101116

2. Name: Faisal Ahmed

ID: 0432320005101129

3. Name: Md. Sayed

ID: 0432320005101148

Submitted To:

Audity Ghosh ,

Department of Computer Science and
Engineering

Declaration

We hereby declare that this project report titled "Car Racing – Obstacle Dodge Drive" has been prepared by us as part of the requirements for the course CSE 358 (Computer Graphics and Multimedia Lab). This project is our own original work and has been developed independently using OpenGL with C++. No part of this report has been submitted elsewhere for any academic purpose. All external libraries and references have been properly acknowledged.

Team Name: Thunder Drivers

Date: May 2026

Abstract

This report presents the design and implementation of a 2D car racing game titled "Car Racing – Obstacle Dodge Drive," developed as a Computer Graphics and Multimedia Lab course project. The game is built using OpenGL 3.3 Core Profile with C++, integrating the GLFW library for window management, GLAD for OpenGL function pointer loading, and GLM for matrix and vector mathematics.

The project demonstrates the practical application of seven key concepts from the OpenGL curriculum: window creation, quad rendering using VAO and VBO, GLSL shader programming, procedural texture generation, 2D transformations (translation, scaling, rotation), orthographic projection, and alpha blending. The game features a player-controlled car navigating a three-lane road while avoiding randomly spawning enemy vehicles, with a custom 7-segment score display, progressive difficulty scaling, and multi-state game flow management.

Keywords: OpenGL, GLFW, GLAD, GLM, GLSL Shader, VAO, VBO, Texture, Transformation, Orthographic Projection, Blending, Car Racing, 2D Game

Table of Contents

Chapter / Section	Title	Page
—	Declaration	1
—	Abstract	2
—	Table of Contents	3
1	Introduction	4
1.1	Background	4
1.2	Project Overview	4
1.3	Motivation	4
1.4	Scope of the Project	4
2	Objectives	5
3	Tools and Technologies	6
4	System Design and Architecture	7
4.1	Application Architecture	7
4.2	Game State Machine	7
4.3	Entity Structure	7
4.4	Coordinate System	7
4.5	Delta-Time Normalization	8
5	OpenGL Concepts Applied	9
6	Implementation Details	11
7	Game Features and Controls	13
8	Source Code Structure	14
9	Results	16
10	Challenges and Solutions	17
11	Conclusion	18
—	References	19

Chapter 1: Introduction

1.1 Background

Computer Graphics is a fundamental branch of Computer Science that deals with generating and manipulating visual content using computational techniques. In academic settings, practical game development projects serve as an excellent medium to understand the underlying mathematics and hardware-level operations of real-time rendering pipelines.

OpenGL (Open Graphics Library) is a cross-platform, industry-standard API used for rendering 2D and 3D vector graphics. It communicates directly with the GPU and is widely used in games, simulations, and visualization software. Learning OpenGL requires understanding the programmable rendering pipeline introduced in OpenGL 3.3 Core Profile, which replaces legacy fixed-function approaches with programmable vertex and fragment shaders written in GLSL (OpenGL Shading Language).

1.2 Project Overview

"Car Racing – Obstacle Dodge Drive" is a 2D arcade-style game where the player controls a blue car on a three-lane road. Enemy cars (red) spawn from the top of the screen and move downward at increasing speeds. The player must dodge these obstacles using the left and right arrow keys. The score increases automatically over time, and the speed of all objects accelerates as the score grows, increasing difficulty progressively.

The game implements four distinct states: Menu, Playing, Paused, and Game Over. Each state has its own rendering logic and input handling. A custom 7-segment digit renderer is used for displaying scores without relying on any font library.

1.3 Motivation

The motivation behind this project is to gain hands-on experience with real-time rendering concepts directly applicable to industry-level game development and graphics programming. By building a complete interactive game from scratch — including shader programming, texture generation, transformation matrices, and collision detection — the developer gains deep understanding of the OpenGL pipeline that cannot be obtained through theory alone.

1.4 Scope of the Project

The project covers the following scope:

- Full OpenGL 3.3 Core Profile rendering pipeline
- GLSL vertex and fragment shader programming
- Procedural texture generation without external image files
- Real-time game loop with delta-time based frame-rate-independent updates
- Collision detection between player and enemy vehicles
- Progressive difficulty scaling with score
- Multi-state game management (Menu, Playing, Paused, Game Over)

Chapter 2: Objectives

2.1 Primary Objectives

1. To implement a fully functional 2D car racing game using OpenGL 3.3 Core Profile in C++.
2. To demonstrate the use of the programmable rendering pipeline by writing custom GLSL vertex and fragment shaders.
3. To apply 2D affine transformations — translation, scaling, and rotation — using GLM matrix operations passed to the GPU.
4. To implement orthographic projection for a fixed-coordinate-system 2D scene.
5. To generate and upload procedural textures to the GPU using `glTexImage2D`, rendering car sprites without any external image files.
6. To implement alpha blending for semi-transparent overlays on pause and game-over screens.
7. To design a complete real-time game loop with delta-time normalization for frame-rate-independent movement.

2.2 Secondary Objectives

8. To implement multi-state game management (Menu, Playing, Paused, Game Over).
9. To design a custom 7-segment digit renderer for score display without any font library.
10. To apply progressive difficulty scaling by increasing object speeds as the score grows.
11. To implement axis-aligned bounding box (AABB) collision detection.
12. To gain familiarity with the GLFW, GLAD, and GLM ecosystem used in OpenGL projects.

2.3 Learning Outcomes

Upon completion of this project, the developers are expected to:

- Understand the full OpenGL rendering pipeline from CPU to screen.
- Write and debug GLSL shaders for both vertex transformation and fragment coloring.
- Manage GPU resources: VAO, VBO, texture objects, and shader programs.
- Apply mathematical transformations to game objects using matrix algebra.
- Structure a real-time application with proper game loop architecture.

Chapter 3: Tools and Technologies

3.1 Development Environment

Tool / Technology	Version	Purpose
C++	C++17	Primary programming language
OpenGL	3.3 Core Profile	Graphics rendering API
GLFW	3.x	Window creation, input handling, OpenGL context management
GLAD	—	OpenGL function pointer loader
GLM	0.9.x	Mathematics library for matrices and vectors
GCC / G++	MSYS2 / Linux APT	C++ compiler
GNU Make	—	Build automation
Windows 10/11 or Linux	—	Target operating system

3.2 Library Details

3.2.1 GLFW

GLFW (Graphics Library Framework) is a lightweight, open-source library for creating windows, handling input events, and managing OpenGL contexts. In this project, GLFW creates a 500x700 pixel window, sets up the OpenGL 3.3 Core Profile context, registers keyboard callbacks, and drives the main render loop via `glfwPollEvents()` and `glfwSwapBuffers()`.

3.2.2 GLAD

GLAD is a loader generator for OpenGL. Since OpenGL function pointers are not available at link time on most platforms, GLAD queries the GPU driver at runtime and populates function pointers. The `glad.c` source file is compiled directly into the project and initialized using `gladLoadGLLoader` with the GLFW proc address function.

3.2.3 GLM (OpenGL Mathematics)

GLM is a header-only C++ mathematics library designed to mirror GLSL syntax and types. It provides `vec2`, `vec3`, `vec4` (vectors), `mat4` (4x4 matrix), and functions such as `glm::translate`, `glm::scale`, `glm::rotate`, `glm::ortho`, and `glm::value_ptr`. All transformation and projection matrices in this project are constructed using GLM and uploaded to the shader using `glUniformMatrix4fv`.

3.3 Build System

The project uses a GNU Makefile with two targets: `make win` for Windows using the GLFW DLL in the `lib/` directory, and `make linux` for Linux linking against the system-installed `libglfw3-dev` package. The compiled executable is placed in the `build/` directory.

Chapter 4: System Design and Architecture

4.1 Application Architecture

The application follows a classic real-time game loop architecture. All logic, update, and rendering operations are performed within a single main loop that runs until the window receives a close signal.

Phase	Operation	Frequency
Initialization	Window creation, GLAD loading, shader compilation, VAO/VBO setup, texture generation	Once at startup
Event Polling	glfwPollEvents() — keyboard state update	Every frame
Update	Physics update, spawn logic, collision detection, score increment	Every frame (PLAYING state only)
Render	glClear, draw road, draw cars, draw HUD, draw overlays	Every frame
Swap	glfwSwapBuffers — present rendered frame	Every frame

4.2 Game State Machine

The game is managed through a finite state machine with four states defined in the GameState enum:

State	Description	Transition To
MENU	Initial state. Shows title screen with car decorations and a blinking start indicator.	PLAYING (SPACE key)
PLAYING	Active gameplay. Physics, spawn, collision, and score are all active.	PAUSED (P key) / GAME_OVER (collision)
PAUSED	Gameplay frozen. Dark overlay with pause icon shown.	PLAYING (P key)
GAME_OVER	Collision detected. Shows rotated crashed car, final score, high score, and restart prompt.	PLAYING (SPACE key)

4.3 Entity Structure

The Player is represented by three floats: playerX, playerY (position), and constants PW=38, PH=62 (half-width and half-height). The player moves horizontally only, constrained between x=138 and x=362. Enemy vehicles are stored as a dynamic std::vector<Enemy>. Each Enemy struct contains: x, y (position), speed (downward velocity), and r, g, b (random warm reddish color tint).

4.4 Coordinate System

The project uses a pixel-space coordinate system with the origin at the bottom-left corner. The x-axis extends right to 500, and the y-axis extends upward to 700. This is established using `glm::ortho(0, 500, 0, 700)` as the projection matrix, computed once and sent to the shader as a uniform.

4.5 Delta-Time Normalization

All velocity values are multiplied by $dt60 = \text{deltaTime} * 60.0$, where `deltaTime` is the elapsed time in seconds between consecutive frames obtained via `glfwGetTime()`. This ensures frame-rate-independent movement regardless of hardware performance.

Chapter 5: OpenGL Concepts Applied

5.1 Concept 1 — Window Creation (GLFW + GLAD)

Reference: LearnOpenGL — Getting Started / Creating a Window

The application window is created using `glfwInit()`, `glfwWindowHint()` to request OpenGL 3.3 Core Profile, and `glfwCreateWindow(500, 700, "Car Racing | OpenGL", NULL, NULL)`. GLAD is initialized via `gladLoadGLLoader` to load all OpenGL function pointers. Two callbacks are registered: `framebuffer_size_callback` for viewport resizing and `keyCallback` for all keyboard input handling.

5.2 Concept 2 — VAO and VBO (Hello Triangle)

Reference: LearnOpenGL — Getting Started / Hello Triangle

A unit quad from (0,0) to (1,1) is defined as two triangles using six vertices. Each vertex contains four floats: a 2D position (x, y) and a 2D texture coordinate (u, v). This data is uploaded to a Vertex Buffer Object (VBO). A Vertex Array Object (VAO) records how to interpret the buffer: attribute 0 is the 2D position and attribute 1 is the texture coordinate. This single unit quad geometry is reused for every rectangular object in the game, scaled and positioned using the model matrix.

5.3 Concept 3 — GLSL Shaders

Reference: LearnOpenGL — Getting Started / Shaders

A single shared sprite shader handles all rendering. The vertex shader applies model and projection matrices:

```
gl_Position = uProjection * uModel * vec4(aPos, 0.0, 1.0);
```

The fragment shader outputs color based on a `uUseTexture` boolean. When texturing is active, the texture sample is multiplied by a `uColor` tint. When disabled, `uColor` is used directly as a flat fill color. This allows the same shader to render both solid-colored primitives and textured car sprites.

5.4 Concept 4 — Textures (Procedural Generation)

Reference: LearnOpenGL — Getting Started / Textures

All car textures are generated programmatically in `makeCarTexture(bool isPlayer)`. A 16x24 pixel RGBA array is constructed in CPU memory by assigning colors per pixel based on car part rules: body color (blue for player, red for enemies), dark grey wheels at all four corners, light blue windscreen in the upper-center region, yellow headlights for the player, and red brake lights for enemies. The pixel array is uploaded to the GPU using `glTexImage2D` with `GL_NEAREST` filtering for a retro pixel art appearance.

5.5 Concept 5 — Transformations (GLM)

Reference: LearnOpenGL — Getting Started / Transformations

Every object is positioned and sized by constructing a 4x4 model matrix in `drawSprite()`. The sequence is: (1) translate the unit quad origin to (cx-hw, cy-hh) so the center aligns at (cx, cy); (2) if rotation is requested, translate to center, apply `glm::rotate` around the Z-axis, translate back; (3) scale by (hw*2, hh*2) to set actual pixel dimensions. This matrix is uploaded via `glUniformMatrix4fv`. The Game Over screen applies an 18-degree rotation to the crashed player car using `glm::rotate`.

5.6 Concept 6 — Coordinate Systems (Orthographic Projection)

Reference: LearnOpenGL — Getting Started / Coordinate Systems

A 2D orthographic projection matrix is created once using `glm::ortho(0.0f, 500.0f, 0.0f, 700.0f)`. This maps pixel coordinates directly to NDC without perspective distortion. The matrix is sent to the shader as `uProjection` and applied in every vertex shader invocation. Because the viewport does not change during gameplay, the matrix is computed once at initialization.

5.7 Concept 7 — Blending (Alpha Transparency)

Reference: LearnOpenGL — Advanced OpenGL / Blending

Alpha blending is enabled with `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`, implementing the standard over-compositing equation. This is used in `drawOverlay()` to render dark semi-transparent rectangles for the Menu (alpha 0.55), Pause (alpha 0.50), and Game Over (alpha 0.65) screens. The blinking SPACE-to-start indicator also uses alpha computed as $\sin(\text{time} * 3) * 0.5 + 0.5$ for a smooth pulsing effect.

Chapter 6: Implementation Details

6.1 Shader Compilation and Linking

The `compileShader()` function creates a shader object, attaches the GLSL source string, compiles it, and checks for errors using `glGetShaderiv(GL_COMPILE_STATUS)`. The `linkProgram()` function creates a program, attaches both shaders, links them, and deletes the individual shader objects. All four uniform locations (`uModel`, `uProjection`, `uColor`, `uUseTexture`) are queried with `glGetUniformLocation()` after linking.

6.2 Quad Geometry

The `buildGeometry()` function defines a unit quad as two triangles (six vertices) with interleaved position and UV data. The stride is 4 floats (16 bytes), position at offset 0, UV at offset 8 bytes. This single VAO and VBO are reused for every shape rendered in the scene.

6.3 Road Scene Rendering

The `drawRoad()` function renders the static scene background: a dark navy sky rectangle in the upper 28% of the screen; green grass strips on both sides of the road; a dark grey road in the center; white border lines at road edges; six scrolling yellow lane-dash rectangles; and six stylized trees using stacked `drawRect` calls on each side of the road.

6.4 Scrolling Lane Animation

Each of the six lane-dash positions in `laneY[]` is decremented by `laneSpeed * dt60` each frame. When any dash goes below `y = -40`, it wraps back by adding 740, creating an infinite scrolling illusion without any actual road object movement.

6.5 Enemy Spawning and Management

A `spawnTimer` is incremented each frame. When it reaches `spawnInterval` (initially 120 frames), a new `Enemy` is created at a random lane `x` position at `y = SCR_HEIGHT + 80` (above screen). Its speed is `baseSpeed + (rand()%8)*0.3f`. After each spawn, `spawnInterval` decreases by 0.4 (minimum 45), causing more frequent spawns over time. Enemies are removed when their `y` drops below -100.

6.6 Collision Detection (AABB)

The `checkCollision()` function uses Axis-Aligned Bounding Box (AABB) intersection testing:

```
fabsf(playerX - e.x) < PW + 26 AND fabsf(playerY - e.y) < PH + 45
```

The constants 26 and 45 represent the approximate half-dimensions of an enemy car. When overlap is detected, the game transitions to `GAME_OVER` and updates the high score if the current score exceeds it.

6.7 Score and Difficulty Progression

A `scoreTimer` is incremented each frame. Every 30 frame-units, the score increments by 1. After each increment, speed values are updated as:

- `baseSpeed = 3.5 + score * 0.065`
- `laneSpeed = 6.5 + score * 0.11` (capped at 28)

New enemies inherit the updated `baseSpeed`, so higher scores produce faster obstacles.

6.8 7-Segment Digit Renderer

The `drawDigit()` function implements a software 7-segment display. A boolean array of 7 segments is set per digit using a switch statement. Each active segment is drawn as a small `drawRect` at the appropriate relative position. The `drawNumber()` function converts an integer to its decimal string and renders each digit spaced by $1.3 * \text{size}$ pixels. This system renders score and high score on the HUD without any font library.

Chapter 7: Game Features and Controls

7.1 Features Summary

Feature	Description
3-Lane Road	Road divided into 3 lanes with a scrolling yellow center dash
Player Car	Blue textured car; moves left and right on keyboard input
Enemy Cars	Red textured cars; spawn randomly in any lane from the top
Progressive Speed	Both cars and road scroll faster as score increases
Score Display	Custom 7-segment renderer shows current score (yellow) and high score (green)
Speed Bar	HUD bar shows current speed ratio; transitions from green to red
Pause System	P key pauses or resumes game; shows pause bars with dark overlay
Game Over Screen	Rotated crashed car, final score, best score, blinking restart prompt
Menu Screen	Title with decorative cars, high score display, blinking SPACE indicator
Blending Effects	Semi-transparent dark overlays for Menu, Pause, and Game Over states
Procedural Textures	All sprites generated in CPU memory; no external image files required
Session High Score	Best score retained for the duration of the current game session

7.2 Keyboard Controls

Key	Action
LEFT Arrow (←)	Move player car to the left
RIGHT Arrow (→)	Move player car to the right
SPACE	Start game from Menu / Restart from Game Over screen
P	Pause the game / Resume from Pause state
ESCAPE	Close the game window

Chapter 8: Source Code Structure

8.1 Project Directory Structure

Path	Description
car racing/src/main.cpp	Main source file (~690 lines) — full game implementation
car racing/src/glad.c	GLAD OpenGL loader source (compiled into project)
car racing/include/	Header files — GLAD, GLFW, GLM library headers
car racing/lib/	GLFW DLL and import library for Windows builds
car racing/build/	Compiled executable (main.exe on Windows)
car racing/Makefile	Build script with win and linux targets
car racing/ReadMe.md	Build instructions for Windows and Linux

8.2 Function Summary

Function	Purpose
compileShader()	Compile a GLSL shader from a source string
linkProgram()	Link vertex and fragment shaders into a shader program
buildGeometry()	Create VAO and VBO for the unit quad
makeCarTexture(bool)	Generate procedural 16x24 RGBA car texture and upload to GPU
drawSprite()	Core render function — applies model matrix, color, optional texture
drawRect()	Shorthand for drawSprite without texture (flat color)
drawCar()	Shorthand for drawSprite with texture and optional rotation angle
drawDigit()	Render a single 7-segment digit
drawNumber()	Render a multi-digit integer using drawDigit
drawRoad()	Render the full road scene (sky, grass, road, dashes, trees)
drawOverlay(float)	Render a semi-transparent dark fullscreen overlay
drawHUD()	Render the HUD bar with score, high score, and speed bar
checkCollision()	AABB collision test between player and a given enemy
spawnEnemy()	Add a new enemy at a random lane at the top of the screen
resetGame()	Reset all game state variables for a new game
keyCallback()	GLFW keyboard event callback handler

main()	Application entry point: initialization, render loop, cleanup
--------	---

Chapter 9: Results

9.1 Achieved Outcomes

The project successfully achieves all stated objectives. The game compiles and runs on Windows 10/11 and Linux (Ubuntu 20.04+) with OpenGL 3.3 support. All seven OpenGL concepts are concretely demonstrated in the working executable.

Requirement	Status	Evidence
Window creation with OpenGL 3.3	Completed	Game window opens correctly
VAO/VBO quad geometry	Completed	All shapes rendered via the unit quad
Custom GLSL shaders	Completed	Single sprite shader handles all rendering
Procedural texture generation	Completed	Car sprites visible; no external images
GLM 2D transformations	Completed	Cars positioned, sized, and rotated correctly
Orthographic projection	Completed	Pixel coordinates map precisely to screen
Alpha blending overlays	Completed	Menu, Pause, and Game Over overlays render correctly
Multi-state game logic	Completed	All 4 states transition correctly
Collision detection	Completed	Game Over triggers reliably on car contact
Progressive difficulty	Completed	Speed increases linearly with score

9.2 Performance

The game runs at the monitor's refresh rate (60–144+ FPS depending on hardware) with negligible CPU and GPU load. Because the entire scene is rendered using a single VAO and a single shader program, draw call overhead is minimal. Delta-time normalization ensures smooth, frame-rate-independent gameplay across different hardware configurations.

Chapter 10: Challenges and Solutions

Challenge	Solution Applied
Setting up GLFW and GLAD on Windows without a package manager	Used MSYS2 for GCC and Make. Placed glfw3.dll in both lib/ and build/ directories. The glad.c loader source was compiled directly into the project to avoid linking issues.
Frame-rate-dependent movement speed	Introduced delta-time ($\text{deltaTime} = \text{currentTime} - \text{lastTime}$) and a dt60 multiplier. All velocities are multiplied by dt60 to normalize movement to a 60 FPS equivalent regardless of actual frame rate.
Displaying scores without a font library	Implemented a custom 7-segment digit renderer. Each digit is drawn using up to 7 small rectangles placed at computed relative positions, requiring no bitmap font or texture atlas.
Rendering car sprites without loading image files	Procedurally generated a 16x24 RGBA pixel array in CPU memory using conditional per-pixel logic. Uploaded via <code>glTexImage2D</code> with <code>GL_NEAREST</code> filtering for a retro pixel art appearance.
Centering the unit quad at an arbitrary position	The model matrix first translates to (cx-hw, cy-hh) so that the quad's origin aligns with the desired center, then scales by (hw*2, hh*2) to set the full pixel dimensions.
Enemy difficulty scaling over time	Both <code>spawnInterval</code> (decreasing per spawn) and <code>enemy speed</code> (linearly increasing with score) are tied to the score variable. The spawn interval decreases to a minimum of 45, and <code>baseSpeed</code> grows as $\text{score} * 0.065$.

Chapter 11: Conclusion

This project successfully demonstrates the implementation of a complete, interactive 2D game using the OpenGL 3.3 Core Profile rendering pipeline in C++. All seven computer graphics concepts — window creation, VAO/VBO geometry, GLSL shader programming, procedural texture generation, GLM transformations, orthographic projection, and alpha blending — are concretely applied in a coherent, functional application.

The "Car Racing – Obstacle Dodge Drive" game provides a practical foundation for understanding real-time rendering at the hardware level. By implementing every system from scratch, including the sprite renderer, procedural texture pipeline, 7-segment score display, game state machine, and collision detection, the team gained comprehensive insight into how graphical applications communicate with the GPU to produce interactive experiences.

The project also demonstrates good software engineering practices: separation of rendering logic from game logic, reuse of a single geometry object for all shapes, delta-time-based frame-rate independence, and clean state machine design for game flow management.

11.1 Future Enhancements

- Audio integration using OpenAL or irrKlang for engine sounds and collision effects
- Multiple difficulty modes (Easy, Medium, Hard) with preset speed profiles
- Power-up items such as speed boost and shield rendered as additional textured sprites
- Multiple player car skins using different procedural texture patterns
- Persistent high score saved to a local file using `std::fstream`
- Smooth lane-changing animation using interpolation instead of instant movement
- Background parallax scrolling with multiple layers at different speeds

References

de Vries, J. (n.d.). Creating a window. LearnOpenGL. <https://learnopengl.com/Getting-started/Creating-a-window>

de Vries, J. (n.d.). Hello triangle. LearnOpenGL. <https://learnopengl.com/Getting-started/Hello-Triangle>

de Vries, J. (n.d.). Shaders. LearnOpenGL. <https://learnopengl.com/Getting-started/Shaders>

de Vries, J. (n.d.). Textures. LearnOpenGL. <https://learnopengl.com/Getting-started/Textures>

de Vries, J. (n.d.). Transformations. LearnOpenGL. <https://learnopengl.com/Getting-started/Transformations>

de Vries, J. (n.d.). Coordinate systems. LearnOpenGL. <https://learnopengl.com/Getting-started/Coordinate-Systems>

de Vries, J. (n.d.). Blending. LearnOpenGL. <https://learnopengl.com/Advanced-OpenGL/Blending>

GLFW Development Team. (2024). GLFW documentation (Version 3.4). <https://www.glfw.org/documentation.html>

GLAD Contributors. (n.d.). GLAD — Multi-language GL/GLES/EGL/GLX/WGL loader-generator. <https://glad.dav1d.de/>

Truc, C. (2023). GLM — OpenGL mathematics library (Version 0.9.9). G-Truc Creation. <https://glm.g-truc.net/0.9.9/>

Khronos Group. (2022). OpenGL 3.3 reference pages. <https://registry.khronos.org/OpenGL-Refpages/gl4/>

Eck, D. J. (2022). Introduction to computer graphics (Version 1.4). <https://math.hws.edu/graphicsbook/>